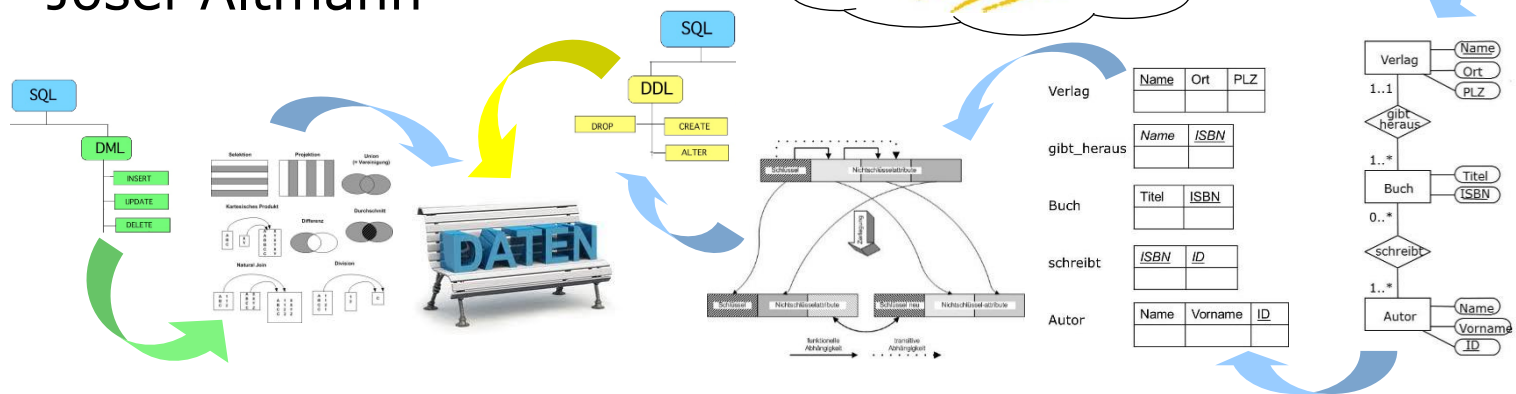


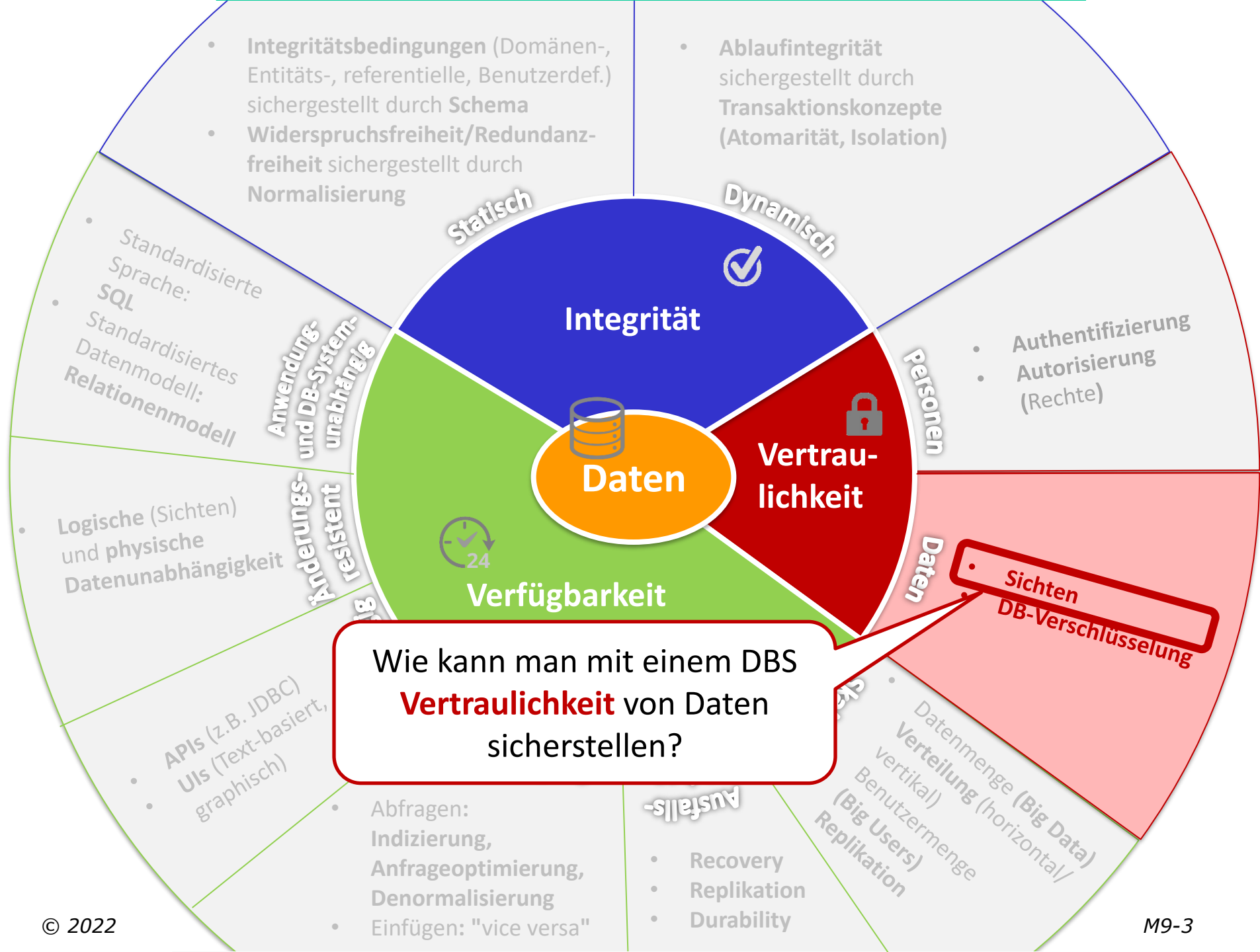
Modul 9

Sichten

Josef Altmann





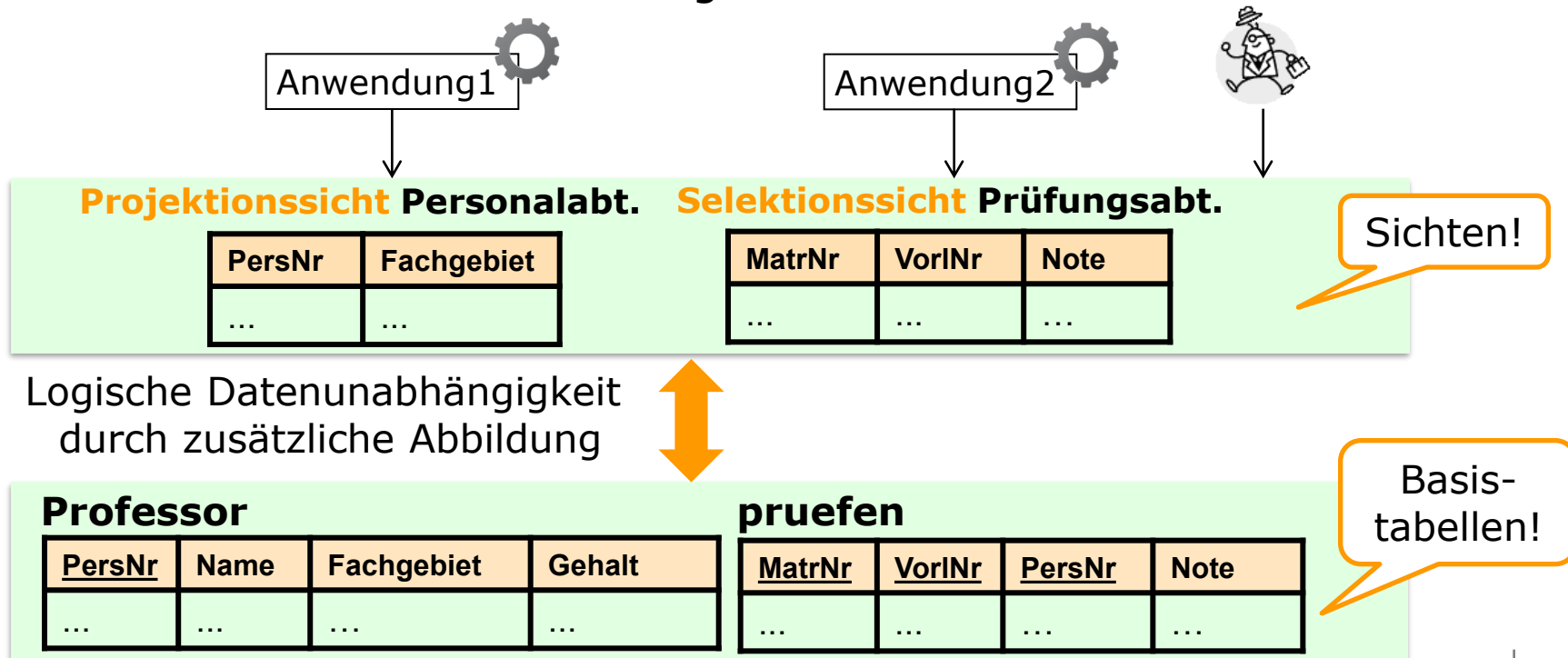


Überblick

- Motivation
- Konzept und Umsetzung in SQL
- Sichtarten
- Einschränkungen bei Änderungsoperationen
- Zusammenfassung

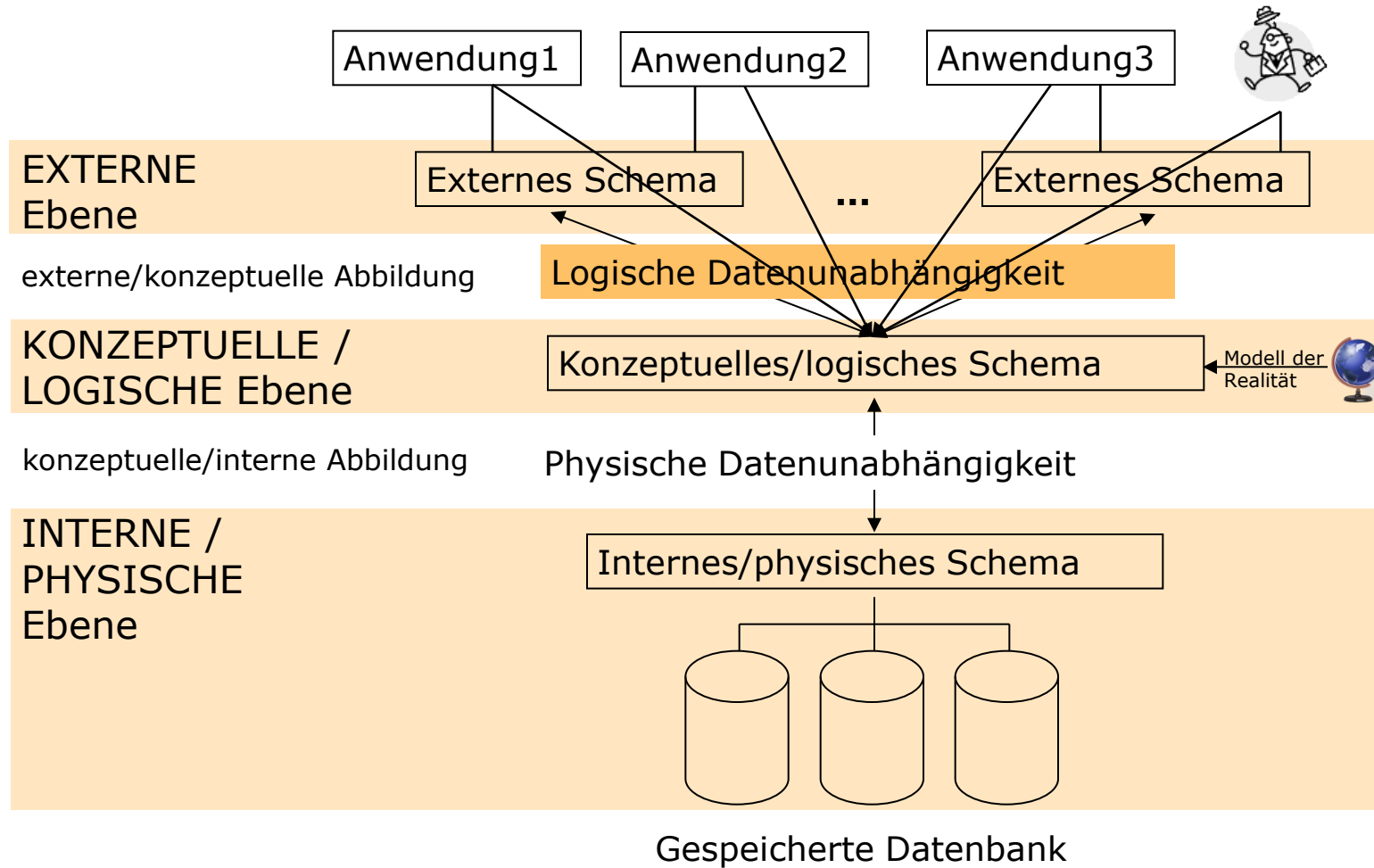
Motivation

- Gesamt-Datenbestand ist angepasst, so dass jede Benutzergruppe/Anwendung nur die Daten sieht, die sie ...
 - **sehen soll** ⇒ Datenschutz, Vertraulichkeitsaspekte
 - **sehen will** ⇒ Übersichtlichkeit, Vereinfachung, leichtere Benutzung



Motivation

ANSI/X3/SPARC: Normungsvorschlag
für Systemarchitektur von DBMS



- Logische Datenunabhängigkeit schützt die Benutzer in gewissen Grenzen vor Veränderungen am Datenbankschema.

Sichten

Was ist eine Sicht?

⇒ Stellt **Teilmengen von Daten** aus einer oder mehreren Tabellen (Basistabellen, Sichten) logisch dar.

└─ = mit **CREATE TABLE** definierte Tabellen

■ Sicht = (normalerweise) **Virtuelle Tabelle**

■ Was bedeutet "**virtuell**"?

- Sicht kann **wie** eine **Basistabelle verwendet** werden
 - `SELECT ... FROM VirtuelleTabelle WHERE ...`
 - `INSERT, DELETE, UPDATE` ⇒ jedoch mit Einschränkungen
- Sicht (virtuelle Tabelle) ist **nicht permanent gespeichert**
- Sicht (virtueller Tabelleninhalt) wird bei **jeder Verwendung** aus anderen Tabellen **neu berechnet**

➔ "**dynamisches Fenster**" auf zugrunde liegende Basistabellen und Sichten

Sichten

Definition

- Sicht **enthält alle Tupel der Basistabelle(n)**, die die Selektionsbedingung erfüllen.
- An die Sicht können **beliebige Anfragen** gestellt werden, auch in Kombination mit anderen Tabellen (Join).
- In Wirklichkeit wird lediglich die **Sichtdefinition** in die **Anfrage eingesetzt** und dann **ausgewertet**:

Beispiel

```
CREATE VIEW pruefenSicht AS
  SELECT MatrNr, VorlNr, PersNr
  FROM pruefen;
```

```
SELECT * FROM pruefenSicht WHERE MatrNr = 24002;
```



```
SELECT * FROM
  (SELECT MatrNr, VorlNr, PersNr FROM pruefen)
WHERE MatrNr = 24002;
```


Motivation

Worin liegen die Vor- und Nachteile?

■ Vorteile

- **Datenzugriff einschränken**
... Sicherheits- und Autorisierungsmechanismus
- **Strukturierung der Datenbank**
... zugeschnitten auf Benutzergruppen
- **Logische Datenunabhängigkeit gewährleisten**
... Sichten stabil bei Änderungen der Datenbankstruktur und umgekehrt bei Änderungen der Anwendungen
- **Komplexe Anfragen erleichtern**
... durch einfache Abfrage der Sicht (~ "Makro")

■ Nachteile

- **Eingeschränkte Änderungen**
... aktualisierbare und nicht-aktualisierbare Sichten

Konzept und Umsetzung in SQL

Definition einer Sicht

- Anfrage in die **CREATE VIEW**-Anweisung einbetten

Notation

```
CREATE [OR REPLACE] VIEW sichthname [ schemadeklaration ]  
AS SQL-Anfrage → legt den Inhalt für die virtuelle Sicht fest (Berechnungsvorschrift)  
[ WITH CHECK OPTION ] ;
```

Konzept und Umsetzung in SQL

Definition einer Sicht

Bereits existierende Rechte
bleiben dadurch erhalten

- Mit der Option **OR REPLACE** können Sichten geändert werden

Notation

```
CREATE OR REPLACE VIEW pruefenSicht AS
  SELECT MatrNr, VorlNr, PersNr AS Personalnummer
  FROM pruefen;
```

- Mit **DROP VIEW** können Sichten gelöscht werden

Notation

```
DROP VIEW pruefenSicht;
```

- Auf eine Sicht kann **nicht mehr zugegriffen werden**, wenn eine **Basistabelle** oder eine Sicht, auf die beim Aufbau der Sicht Bezug genommen wurde, **entfernt** bzw. **geändert** wurde

Konzept und Umsetzung in SQL 1/4

Anpassung an unterschiedliche Benutzergruppen

Beispiel

Aus Datenschutzgründen soll im Allgemeinen nur die Prüfungsliste, nicht aber die Note lesbar sein:

```
CREATE VIEW pruefenSicht AS
  SELECT MatrNr, VorlNr, PersNr AS Personalnummer
  FROM pruefen;
```

Konzept und Umsetzung in SQL 2/4

Vereinfachung von Anfragen

Beispiel

Vermeidung der ständigen Formulierung des Joins
Studenten hören Vorlesungen Professoren

```
CREATE VIEW StudProfSicht (Sname, Semester, Titel, Pname) AS
SELECT s.Name, s.Semester, v.Titel, p.Name
FROM Studenten s INNER JOIN hoeren h USING (MatrNr)
INNER JOIN Vorlesungen USING (VorlNr)
INNER JOIN Professoren ON (v.gelesenVon = p.PersNr);
```

Anwendung StudProfSicht

Suchen Sie Name und Semester aller Studierenden von Sokrates?

```
SELECT DISTINCT Sname, Semester
FROM StudProfSicht
WHERE Pname = 'Sokrates');
```

Konzept und Umsetzung in SQL 3/4

Verdichten und anonymisieren von Daten

Beispiel

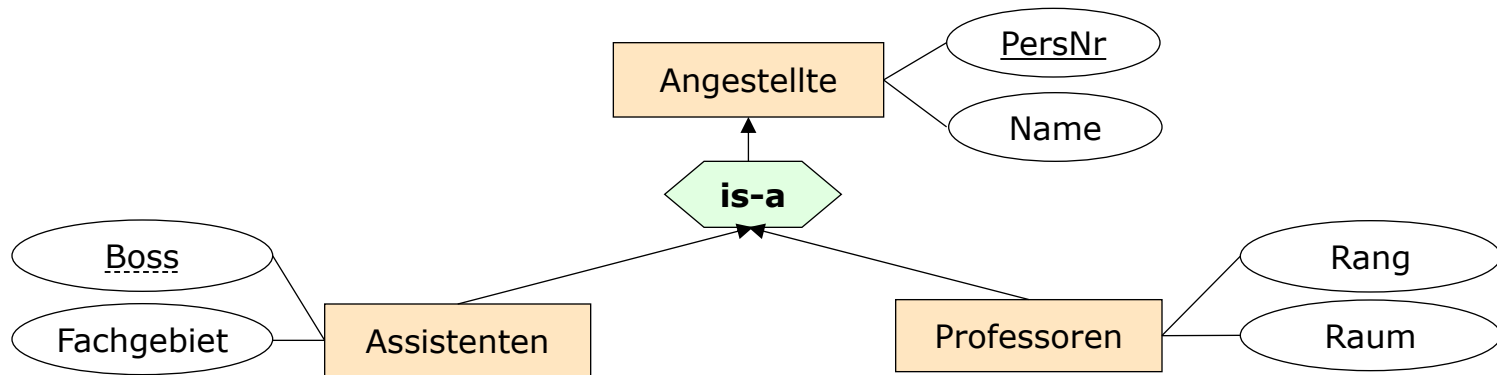
Ermittle die Durchschnittsnote aller bisher abgenommenen Prüfungen der Professoren. Scheide jene Professoren aus, die bislang weniger als 50 Prüfungen abgenommen haben.

```
CREATE VIEW PrueferHaerte(Name, HaerteGrad) AS
SELECT prof.Name, AVG(pruef.Note)
FROM Professor prof      INNER JOIN pruefen pruef
                        ON (prof.PersNr = pruef.PersNr)
GROUP BY prof.Name, prof.PersNr
HAVING COUNT(*) >= 50;
```

→ Anonymität der Prüfungsdaten wird gewährleistet!

Konzept und Umsetzung in SQL 4/4

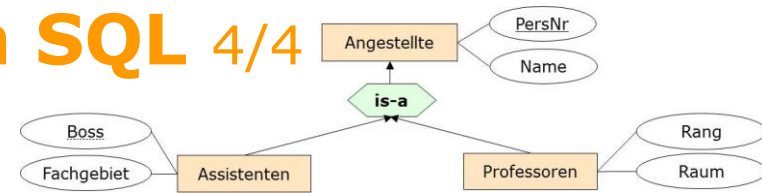
Realisierung von Generalisierung



- Sichten realisieren Vererbung:
 - Tupel des Untertyps sollen auch automatisch zum Obertyp gehören
 - Attribute des Obertyps sollen automatisch vererbt werden
- entweder Untertypen oder Obertypen als Sicht definieren
- Entscheidung aufgrund der Häufigkeit der Zugriffe

Konzept und Umsetzung in SQL 4/4

Realisierung von Generalisierung



Beispiel (Untertypen als Sicht)

```

CREATE TABLE Angestellte (PersNr INTEGER PRIMARY KEY,
                           Name VARCHAR (30) NOT NULL);

CREATE TABLE ProfDaten ( PersNr INTEGER PRIMARY KEY,
                           Rang CHAR(2) CONSTRAINT Check_Rang
                           CHECK(Rang IN ('C2', 'C3')),
                           Raum INTEGER UNIQUE);

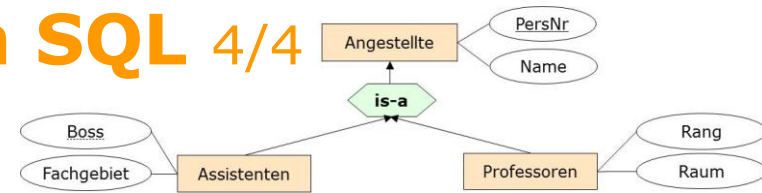
CREATE TABLE AssiDaten ( PersNr INTEGER PRIMARY KEY,
                           Fachgebiet VARCHAR (30),
                           Boss INTEGER,
                           CONSTRAINT FK_Assistenten_Boss
                           FOREIGN KEY (Boss) REFERENCES Professoren(PersNr)
                           ON DELETE SET NULL);

CREATE VIEW Professoren AS
  SELECT * FROM Angestellte NATURAL JOIN ProfDaten;

CREATE VIEW Assistenten AS
  SELECT * FROM Angestellte NATURAL JOIN AssiDaten;
  
```


Konzept und Umsetzung in SQL 4/4

Realisierung von Generalisierung



Beispiel (Obertypen als Sicht)

```

CREATE TABLE Professoren ( PersNr INTEGER PRIMARY KEY,
                             Name VARCHAR (30) NOT NULL,
                             Rang CHAR(2) CONSTRAINT Check_Rang
                                CHECK(Rang IN ('C2', 'C3')),
                             Raum INTEGER UNIQUE);

CREATE TABLE Assistenten ( PersNr INTEGER PRIMARY KEY,
                             Name VARCHAR (30) NOT NULL
                             Fachgebiet VARCHAR (30),
                             Boss INTEGER,
                             CONSTRAINT FK_Assistenten_Boss
                             FOREIGN KEY (Boss) REFERENCES Professoren(PersNr)
                             ON DELETE SET NULL);

CREATE VIEW Angestellte AS
  SELECT PersNr, Name FROM Professoren
  UNION
  SELECT PersNr, Name FROM Assistenten;
  
```

Konzept und Umsetzung in SQL

Kriterien für Datenmanipulation über Sichten

- Effektkonformität
 - Benutzer sieht Effekt **als wäre die Änderung auf der Sichtrelation direkt ausgeführt worden**
 - Minimalität
 - Basisdatenbank sollte nur **minimal geändert werden**, um den erwähnten Effekt zu erhalten
 - Konsistenzerhaltung
 - Änderung einer Sicht darf zu **keinen Integritätsverletzungen** der Basistabellen führen
 - Respektierung der Zugriffskontrolle
 - Wird die Sicht aus Gründen der Zugriffskontrolle eingeführt, **darf der bewusst ausgeblendete Teil der Basistabellen von Änderungen der Sicht nicht betroffen werden**
- ⇒ Frage: Inwieweit ist Einfügen, Löschen und Ändern von Tupel in einer virtuellen Sicht erlaubt?

Überblick

- Motivation
- Konzept und Umsetzung in SQL
- Sichtarten
- Einschränkungen bei Änderungsoperationen
- Zusammenfassung

Sichtarten

... bestimmen erlaubte DML-Operation

■ Vier verschiedene Sichtarten

- Projektion $\Rightarrow$ **Projektionssicht**
- Selektion $\Rightarrow$ **Selektionssicht**
- Verbund $\Rightarrow$ **Verbundssicht**
- Aggregation $\Rightarrow$ **Aggregationssicht**

■ Faustregel

- Auf einer Sicht sind nur solche DML-Operationen zulässig, zu denen eine **eindeutige Transformation auf die Basisrelationen** existiert.

Sichtarten

Beispielszenario

Mitarbeiter

<u>PersNr</u>	Gehalt	<u>Abteilung</u>
...	...	...

Abteilung

<u>AbtNr</u>	AbtName	AbtLeiter
...		...

- **Mitarbeiter ...** speichert Daten über Zugehörigkeit von MitarbeiterInnen zu Abteilungen und deren jeweiliges Gehalt
- **Abteilung...** speichert für jede Abteilung den/die LeiterIn

Sichtarten

Projektionssicht 1/3

Mitarbeiter

<u>PersNr</u>	Gehalt	<u>Abteilung</u>
...	...	...

Abteilung

<u>AbtNr</u>	AbtName	AbtLeiter
...		...

Sichten

Projektionssicht $:= \pi_{\text{PersNr}, \text{Abteilung}}(\text{Mitarbeiter})$

- In SQL mit **CREATE VIEW**-Anweisung:

```
CREATE VIEW Projektionssicht AS
SELECT PersNr, Abteilung FROM Mitarbeiter;
```

- Einfügeanweisung für die Sicht ProjektionsSicht:

```
INSERT INTO Projektionssicht VALUES (100, 1);
```

- Korrespondierende Anweisung auf Basisrelation Mitarbeiter:

```
INSERT INTO Mitarbeiter VALUES (100, NULL, 1);
```

- **Konsistenzerhaltung** verletzt, falls Gehalt als NOT NULL definiert wurde

Sichtarten

Projektionssicht 2/3

Mitarbeiter

<u>PersNr</u>	Gehalt	<u>Abteilung</u>
...	...	...

Abteilung

<u>AbtNr</u>	AbtName	AbtLeiter
...		...

Sichten

- Lösch- und Änderungsanweisung für MitarbeiterSicht:
`DELETE FROM Projektionssicht WHERE PersNr = ...;`
- Korrespondierende Anweisung auf Basisrelation Mitarbeiter:
`DELETE FROM Mitarbeiter WHERE PersNr = ...;`
- Problem falls **nicht alle Schlüsselattribute** der Basisrelation in der Sicht vorkommen

Sichtarten

Projektionssicht 3/3

- Probleme bei **Duplikatelimination** (`SELECT DISTINCT`) in der Sichtdefinition:
 - Keine eindeutige Zuordnung zwischen Tupel der Sicht und der Basistabelle möglich!
- Probleme bei **skalaren Ausdrücken** in der `SELECT`-Klausel in der Sichtdefinition:
 - Rückrechnung wäre erforderlich!

```
CREATE VIEW P AS SELECT 3*x*x+x+2*x*x+x+1 AS y FROM A;
```

- Änderungsanweisung wäre bsp. problematisch

```
UPDATE P SET y = 100;
```

- Womit müsste x besetzt werden?
 - Mit der Nullstelle des Polynoms $f(x) = 3x^3 + 2x^2 + x + 1$
Nullstellensuche kein triviales mathematisches Problem
- Kein `INSERT/DELETE/UPDATE` bei `DISTINCT/skalaren` (arithmetischen) Ausdrücken

Sichtarten

Selektionssicht

Sichten

Mitarbeiter

<u>PersNr</u>	Gehalt	<u>Abteilung</u>
...	...	...

Abteilung

<u>AbtNr</u>	AbtName	AbtLeiter
...		...

Selektionssicht := $\sigma_{\text{Gehalt} > 20}(\pi_{\text{PersNr}, \text{Gehalt}}(\text{Mitarbeiter}))$

- In SQL mit **CREATE VIEW**-Anweisung:

```
CREATE VIEW Selektionsicht AS
SELECT PersNr, Gehalt FROM Mitarbeiter WHERE Gehalt > 20000;
```

- Änderungsoperation wäre problematisch: Seiteneffekt

```
UPDATE Selektionssicht SET Gehalt = 15000 WHERE PersNr=100;
```

- Tupelmigration: Der Tupel `Mitarbeiter(100, 15000, 1)` wird aus der Sicht "herausbewegt", da für diesen Tupel die Selektionsbedingung nicht mehr erfüllt ist
- **Effektkonformität / Zugriffskontrolle** verletzt!

Sichtarten

Selektionssicht – Kontrolle der Tupelmigration

- Tupelmigration ist **manchmal erwünscht**
 - Mitarbeiter wechselt den zuständigen Sachbearbeiter, jeder Sachbearbeiter arbeitet mit "seiner" Sicht.
- Tupelmigration ist **manchmal unerwünscht**
 - Zugriffskontrolle
- Deshalb in SQL folgende Möglichkeit:

```
CREATE VIEW Selektionssicht AS
SELECT PersNr, Abteilung
FROM Mitarbeiter
WHERE Gehalt > 20000
WITH CHECK OPTION;
```

- Die Klausel **WITH CHECK OPTION** gibt an, dass INSERT- und UPDATE-Operationen, die über die Sicht durchgeführt werden, nur Zeilen erstellen können, welche die Sicht selektieren kann.

Sichtarten

Verbundssicht 1/2

Sichten

Mitarbeiter

<u>PersNr</u>	Gehalt	<u>Abteilung</u>
...	...	...

Abteilung

<u>AbtNr</u>	AbtName	AbtLeiter
...		...

Verbundsicht := Mitarbeiter ⋈ Abteilung

- In SQL mit **CREATE VIEW**-Anweisung:

```
CREATE VIEW Verbundsicht AS
  SELECT PersNr, Gehalt, AbtNr, AbtLeiter
  FROM Mitarbeiter INNER JOIN Abteilung
  ON (Abteilung = AbtNr);
```

- Änderungsoperation in der Regel nicht eindeutig übersetzbar

```
INSERT INTO Verbundsicht
VALUES (101, 30000, 1, 100);
```

Sichtarten

Verbundssicht 2/2

Sichten

Mitarbeiter

<u>PersNr</u>	Gehalt	<u>Abteilung</u>
...	...	...

Abteilung

<u>AbtNr</u>	AbtName	AbtLeiter
...		...

- Änderung wird transformiert zu

```
INSERT INTO Mitarbeiter VALUES (101, 30000, 1);
```

- plus

- Einfügeanweisung auf Abteilung:

```
INSERT INTO Abteilung VALUES (1, 'IT', 100);
```

- oder alternativ:

```
UPDATE Abteilung SET AbtNr = 1  
WHERE Leiter = 100;
```

- besser bzgl. Minimalitätsforderung
- widerspricht aber Effektkonformität!

- Verbundsichten nicht änderbar!

Sichtarten

Aggregationssicht

Mitarbeiter

<u>PersNr</u>	Gehalt	<u>Abteilung</u>
...	...	...

Abteilung

<u>AbtNr</u>	Leiter
...	...

Sichten

```
CREATE VIEW Aggregationssicht (Abteilung, SummeGehalt) AS
SELECT Abteilung, SUM(Gehalt)
FROM Mitarbeiter
GROUP BY Abteilung;
```

- Folgende Änderung ist nicht eindeutig umsetzbar:

```
UPDATE Aggregationssicht
SET SummeGehalt = SummeGehalt + 1000
WHERE Abteilung = 1;
```

Sichtarten

Klassifikation der Problembereiche

- Verletzung der Schemadefinition/Integrität
 - z.B. Einfügen von `NULL`-Werten bei Projektionssichten
- Verletzung der Zugriffskontrolle
 - Seiteneffekte auf nicht-sichtbaren Teil der Datenbank vermeiden (Tupelmigration, Selektionssichten)
- Nicht immer eindeutige Transformation
 - Auswahlproblem
- Aggregationssichten
 - keine sinnvolle Transformation möglich
- Elementare Sichtänderung soll genau einer atomaren Änderung auf Basisrelation entsprechen
 - 1:1-Beziehung zwischen Sicht-Tupel und Tupel der Basisrelation
 - kein Herausprojizieren von Schlüsseln

Änderungsoperationen

Einschränkungen in SQL:1999

- SQL:1999 und SQL:2003 heben einige der strikten Einschränkungen für Sichtänderungen auf
 - wenn spezielle Voraussetzungen beachtet werden
- Ein `INSERT` sowie ein `UPDATE` ist auf einer **Verbund Sicht** erlaubt,
 - wenn die Sicht über **Primär-Fremdschlüsselverbund** gebildet wurde.

Sichtarten

Herausfinden zulässiger Änderungsoperationen

- **Sichtdefinitionen** sind im **Data Dictionary** abgelegt; ebenso die darauf **zulässigen Änderungsoperationen**
- Diese können mit folgendem SQL-Statement abgefragt werden:

Beispiel

```
SELECT * FROM
USER_UPDATABLE_COLUMNS
WHERE TABLE_NAME= 'viewname';
```



	OWNER	TABLE_NAME	COLUMN_NAME	UPDATABLE	INSERTABLE	DELETABLE
1	P24854	TEST	ACTOR_ID	YES	YES	YES
2	P24854	TEST	FIRST_NAME	YES	YES	YES
3	P24854	TEST	LAST_NAME	YES	YES	YES
4	P24854	TEST	LAST_UPDATE	YES	YES	YES

Änderungsoperationen

Verhindern mit **WITH READ ONLY**

- **WITH READ ONLY**-Option in der Sichtdefinition verhindert, dass Änderungsoperationen ausgeführt werden.

```
CREATE VIEW Verbundsicht AS
  SELECT PersNr, Gehalt, AbtNr, AbtLeiter
  FROM Mitarbeiter INNER JOIN Abteilung
  ON (Abteilung = AbtNr)
  WITH READ ONLY;
```

- Versuch, eine Änderungsoperation (**INSERT**, **UPDATE**, **DELETE**) an einer Zeile in der Sicht auszuführen, führt zu einem Fehler.

Überblick

- Motivation
- Konzept und Umsetzung in SQL
- Sichtarten
- Einschränkungen bei Änderungsoperationen
- Zusammenfassung

Zusammenfassung

- Eigenschaften von Sichten
 - Sicht kann wie eine Tabelle behandelt werden
 - Sichtsemantik: "dynamisches Fenster" auf zugrunde liegende Basistabellen
 - Sichten auf Sichten sind möglich
 - eingeschränkte Änderungen: aktualisierbare und nicht-aktualisierbare Sichten
- Unterschied zwischen Sicht und Tabelle
 - keine Integritätsbedingungen
- Sichtkonzept lässt sich für folgende Zwecke einsetzen
 - logische Datenunabhängigkeit
 - vorformulierte Anfragen
 - Sichten als "Zwischentabellen"
 - Datenschutz

Anhang: Definitionen

Sicht

Virtuelle oder materialisierte Relation, die aus den gespeicherten Basisrelationen der Datenbank mit einer Anfrage berechnet wird.

Logische Datenbankunabhängigkeit

Stabilität der auf Sichten arbeitenden Anwendungsprogramme gegen Änderungen an der Datenbankstruktur.

Sichtdefinition

Definition des Relationenschemas der virtuellen/materialisierten Relation und Definition der Anfrage als Berechnungsvorschrift.

Projektionssicht

Sicht, die mithilfe einer Projektion berechnet wird; sie ist mit Einschränkungen änderbar.

Selektionssicht

Sicht, die mithilfe einer Selektion berechnet wird; sie ist mit Einschränkungen änderbar.

Anhang: Definitionen

WITH CHECK Option

Auffassen der Selektionssicht als Integritätsbedingung an die Sicht, etwa aus Datenschutzgründen: Geänderte Tupel dürfen die Sicht nicht verlassen.

Verbundsicht

Sicht, die mithilfe eines Verbundes berechnet wird; sie ist in SQL-92 nicht änderbar, in SQL:2003 mit Einschränkungen änderbar.

Aggregationssicht

Sicht, die mithilfe von Aggregatfunktionen berechnet wird; sie ist nicht änderbar.

Effektkonformität

Änderung auf der Sicht wirkt so, als sei sie direkt auf der virtuellen Relation realisiert worden.

Minimalität

Änderung auf der Sicht, so dass die Basisrelationen nur minimal verändert werden.